T-Tang

IIC 通信协议的Verilog实现

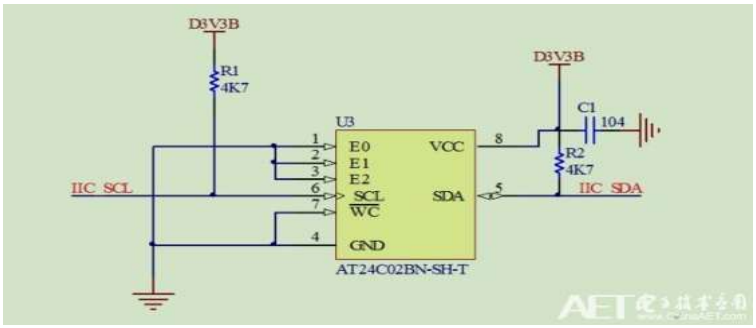
发表于 2016/10/17 23:57:43 阅读（4645） 评论（6）

2 赞

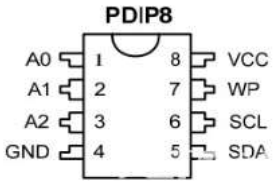
刚刚花了几天时间把 IIC 总算搞懂了一些，查了很多资料，也纠结过于很多细节，不过只要耐着性子，一点点的去理解，去尝试，终会得到你想要的结果，人生不也是吗，嘿嘿~，不闲扯了，下面就写写我的理解以及方法，算是一个总结~

IIC 即 Inter-Integrated Circuit (集成电路总线），这种总线类型是由飞利浦半导体公司在八十年代初设计出来的一种简单、双向、二线制、同步串行总线，主要是用来连接整体电路(ICS)，IIC是一种多向控制总线，也就是说多个芯片可以连接到同一总线结构下，同时每个芯片都可以作为实时数据传输的控制源。这种方式简化了信号传输总线接口。（来源于百度百科）

采用 EEPROM 来验证 IIC 时序，我用的是AT24C02（原理图如下），IIC 时序有单字节写、单字节读，和页写、页读，这次先写单字节写和读，页写和读下次再写。



AT24C02 原理图



AT24C02 封装图

首先看一下各引脚定义：

1，A0、A1、A2为 AT24C02 的片选信号，由于 IIC 总线可以挂载多个 IIC 接口器件，所以每个器件都应该有自己的“身份标识”，通过对 A0,A1,A2输入不同的高低电平，就可以设置该 EEPROM 的片选信号。

2，WP 为读写使能信号，当 WP 悬空或者接地，EEPROM 可读可写，当 WP 接电源，EEPROM 只能读不能写。

3， SCL 为 IIC 接口的时钟线。

4， SDA 为 IIC 接口的数据线。

所以由原理图可以知道，IIC 协议有两条线，一条时钟线SCL，一条双向的数据线SDA，

下面来看一下 IIC 协议的时序图：

写时序：

作者



T-Tang

关注

文章:4 篇
阅读:12317 次
评论:25 次

标签

IIC 通信协议 Verilog FPGA

相关文章

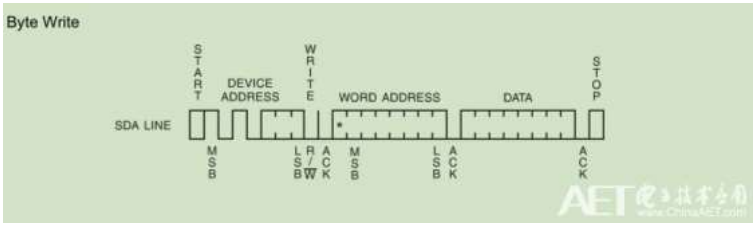
基于FPGA的PCI总线接口多通道DMA控制
赛灵思全球首个28nm FPGA Kintex-7 32
基于FPGA的卷积码编译码器
基于RTL综合策略的状态机优化方案
合作研究取得成功，汽车、工业和通信
SDSoC开发环境能为您带来什么

相关资料

利用 Virtex-5 LXT 器件实现集成化视
基于FPGA的图像边缘检测系统的设计
基于nRF24L01和Actel FPGA的智能探测
多队列FIFO——支持网络QoS的重要芯片
飞思卡尔Verilog经典学习资料
可编程器件固件开发的现代化方法

相关视频

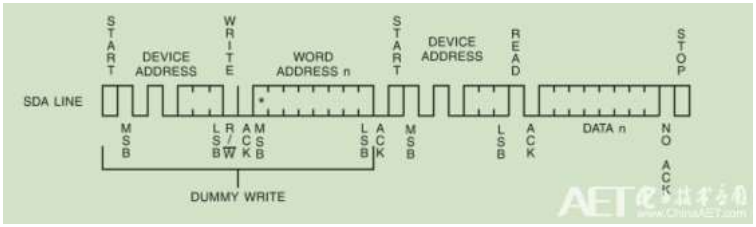
【视频】在下一代定制设计中使用软核M
【视频】PlanAhead设计分析工具学习系
【视频】使用PlanAhead进行RTL与IP的
【视频】使用PlanAhead设计保存——Pl
【视频】采用赛灵思设计平台的ADI FMC
【视频】植物识别（第三届OpenHW开源



由时序图可以看出，如果我们要向 EEPROM 写入一个字节，那么必须经过以下步骤：

1. 发送启动信号
2. 发送器件地址 8'b1010_0000
3. 接收并检测 EEPROM 发来的应答信号 ACK
4. 发送字地址
5. 接收并检测 EEPROM 发来的应答信号 ACK
6. 发送 8bit 有效数据
7. 接收并检测 EEPROM 发来的应答信号 ACK
8. 发送停止信号

读时序：



由时序图可以看出，如果我们要从 EEPROM 读出一个字节，那么必须经过以下步骤：

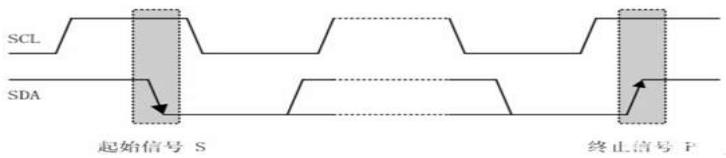
1. 发送启动信号
2. 发送器件地址 8'b1010_0000
3. 接收并检测 EEPROM 发来的应答信号 ACK
4. 发送字地址
5. 接收并检测 EEPROM 发来的应答信号 ACK
6. 发送启动信号
7. 发送器件地址 8'b1010_0001
8. 接收并检测 EEPROM 发来的应答信号 ACK
9. 读取一个字节数据
10. 发送非应答信号 NO ACK
11. 发送停止信号

*** 说明及注意事项：**

（1） 时钟线 SCL：在3.6V以内的系统中，EEPROM 兼容400kHZ 的通信速率，我用的是100kHZ；

（2） 数据线 SDA：SDA 实现双向串行数据传输，该引脚为开漏输出，可与其它多个开漏输出器件或开集电极器件线或连接；

（3） 启动信号和停止信号：在时钟 SCL 的高电平期间对 SDA 拉低或拉高操作，就分别表示启动和停止；



（4） 器件地址： 器件地址有八位，高四位为 1010，低四位中，第 3~1 位为 A2,A1,A0 这 3 位跟实际电路中的硬件有关，AT24C02 芯片中有 A2,A1,A0 这 3 个引脚，所以这 3 位按照实际的原理图来给的。在实际原理图中我们直接把这 3 个引脚接地，所以第 3~1 位为 0，接下来就是第 0 位，第 0 位是 R/W，第 0 位为 1 表示是读操作，第 0 位为 0 表示为写操作。所以

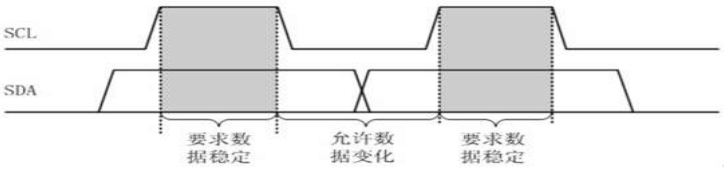
写 为： 8’ b1010_0000;
读 为： 8’ b1010_0001;

（5） 字地址：AT24C02 内部存储空间为 256*8 bit，所以地址位有八位；

（6） 应答信号：应答信号是由接收方发出的，传输完一个字节数据后，主机释放 SDA 控制权， 在 SCL 为高电平期间，如果监测到 SDA 为低电平，说明有应答信号；

（7） 非应答信号：非应答信号是在读完数据后由主机（FPGA）发出的，当 SCL 为高电平期间，如果 SDA 为高电平，说明有非应答信号；

（8） 字节发送：IIC 协议必须要主机取的数据线 SDA 的控制权后，在时钟 SCL 低电平期间一个字节一个字节的传输，高电平期间保持稳定，且先传输最高位（MSB）,每一个字节后面都要有一个应答信号（应答信号是由从机发送的，所以需要先释放数据线 SDA 的控制权）

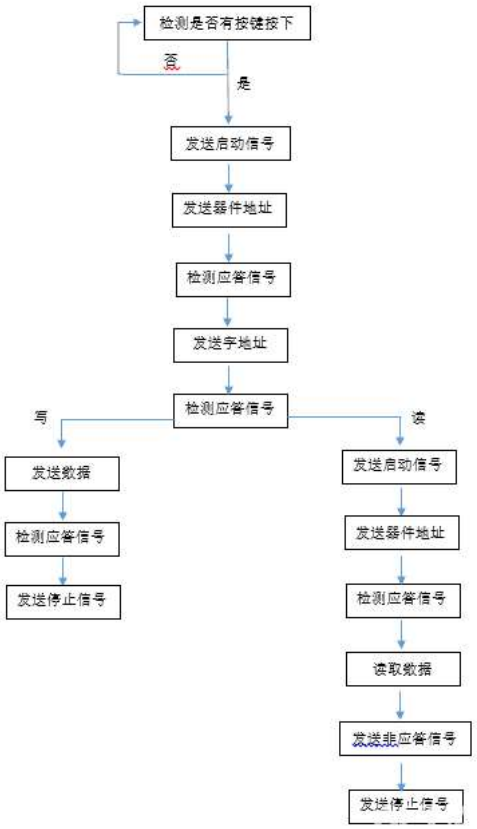


（9） 字节读取：读取数据的时候主机先要释放数据线 SDA 的控制权，读的时候在时钟 SCL 的高电平期间（信号稳定）采样读取，读完后再取得数据线 SDA 的控制权，然后拉高 SDA，即发送非应答信号；

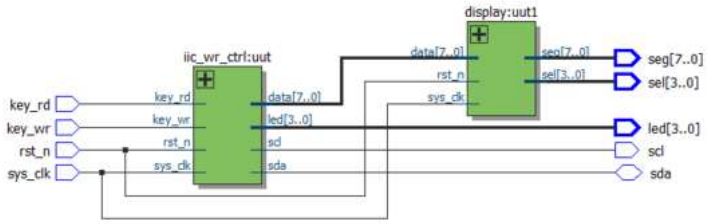
（10）数据线 SDA 在 FPGA 为接收方时，要设置为高阻态；

（11）关于控制权：谁发送，谁就先获得控制权，总线开关拉高，代表主机获得控制权，拉低相反；

* 结合写和读时序，发现1~5步骤是一样的，所以可以这样设置 IIC 读写控制器：



* 代码完成后综合得到的RTL视图如下：



可以根据此模块图进行设计，通过两个按键控制 ill_wr_ctrl （协议读写控制模块），先将数据写入 EEPROM，再将读出来的数据给到数码管显示模块，以此验证协议的正确性

* 这里我主要写一下协议读写控制模块（ill_wr_ctrl），通过仿真来验证

```
module iic_wr_ctrl
(
   input    sys_clk,          //系统时钟
   input    rst_n,            //系统复位信号
   input    key_wr,           //写按键信号
   input    key_rd,           //读按键信号
   inout    sda,              //EEPROM串行双向数据信号
   output   reg scl,          //EEPROM串行时钟信号
   output   reg[7:0] data,    //数据采集信号寄存器
   output   reg[3:0]led       //写入指示信号
);

   reg      [6:0]cnt;         //分频计数器
   reg      [3:0]count;       //数据移位寄存器
   reg      clk;              //协议触发时钟信号
   reg      wr;               //写标志信号
   reg      rd;               //读标志信号
   reg      link_sda;         //总线开关
   reg      sda_buf;          //总线数据寄存器
   reg      [7:0] eeprom_data;//预发送数据
   reg      [3:0] state;      //状态寄存器

   //--------------------------------------------------------------------
   //取得总线控制权时，总线信号就等于sda_buf，释放总线控制权后，将总线设置为高阻态
   assign   sda = (link_sda)? sda_buf : 1'bz;

   //--------------------------------------------------------------------
   //分频计数，产生clk状态跳转时钟
   always@(posedge sys_clk or negedge rst_n)
   begin
      if(!rst_n)
         begin
            cnt <= 0;
            clk <= 0;
         end
      else if(cnt < 125)
            cnt <= cnt + 1;
      else
         begin
            cnt <= 0;
            clk <= ~clk;
         end
   end

   //--------------------------------------------------------------------
   //用clk触发，产生EEPROM串行时钟信号
   always@(negedge clk or negedge rst_n)
   begin
      if(!rst_n)
         scl <= 0;
      else
         scl <= ~scl;
   end

   //--------------------------------------------------------------------
   //利用状态机控制写和读的状态转换
   always@(posedge clk or negedge rst_n)//用clk触发，可以保证在scl高电平或者
   begin                                //低电平中间的稳定期进行数据的采集或变化
      if(!rst_n)
         begin
            state <= 0;
            wr <= 0;
            rd <= 0;
            count <= 0;
            link_sda <= 0;
            sda_buf <= 0;
            eeprom_data <= 0;
            data <= 0;
            led <= 4'b0000;
         end
```

```
else
   begin
      case(state)
         0: begin
               if(!key_rd)
                  rd <= 1;
               else if(!key_wr)
                  wr <= 1;
               else if(!scl && (rd || wr))
                  begin
                     link_sda <= 1;
                     sda_buf <= 1;
                     state <= 1;
                  end
               else
                  state <= 0;
            end
          1: begin//发送启动信号
               if(scl)
                  begin
                     sda_buf <= 0;
                     state <= 2;
                     eeprom_data <= 8'b1010_0000;
                  end
               else
                  state <= 1;
             end

         2: begin//发送器件地址
               if(!scl && count < 8)
                  begin
                     count <= count + 1;
                     eeprom_data <= {eeprom_data[6:0], eeprom_data[7]};
                     sda_buf <= eeprom_data[7];
                  end
               else if(count == 8 && !scl)
                  begin
                     count <= 0;
                     link_sda <= 0;
                     state <= 3;
                  end
               else
                  state <= 2;
            end
         3: begin//检测应答信号
               if(scl)
                  begin
                     if(!sda)
                        begin
                           state <= 4;
                           eeprom_data <= 8'b0000_0011;
                        end
                  end
            end

         4: begin//发送字地址
               link_sda <= 1;
               if(count < 8 && !scl)
                  begin
                     count <= count + 1;
                     eeprom_data <= {eeprom_data[6:0],eeprom_data[7]};
                     sda_buf <= eeprom_data[7];
                  end
               else if(count == 8 && !scl)
                  begin
                     count <= 0;
                     state <= 5;
                     link_sda <= 0;
                  end
               else
                  state <= 4;
            end
         5: begin//检测应答信号，以及判断是写还是读信号
               if(scl)
                  begin
                     if(!sda)
                        begin
                           if(wr == 1)
                              begin
                                 state <= 6;
                                 eeprom_data <= ~8'b0100_1111;// 数码管显示3
                              end

                           else if(rd == 1)
                              begin
                                 state <= 9;
                              end
                        end
                  end
            end
         6: begin//写入数据
               link_sda <= 1;
               if(!scl && count < 8)
                  begin
                     count <= count + 1;
                     eeprom_data <= {eeprom_data[6:0],eeprom_data[7]};
                     sda_buf <= eeprom_data[7];
                  end
               else if(!scl && count == 8)
                  begin
                     count <= 0;
                     link_sda <= 0;
                     state <= 7;
                  end
               else
                  state <= 6;
            end
```

```
            7: begin//检测应答信号
                  if(scl)
                     begin
                        if(!sda)
                              state <= 8;
                     end
               end
            8: begin//发送停止信号
                  link_sda <= 1;
                  sda_buf <= 0;
                  if(scl)
                     begin
                        sda_buf <= 1;
                        led <= 4'b0001;
                        wr <= 0;
                        state <= 0;
                     end
                  else
                     state <= 8;
               end
            9: begin//取得总线控制权
                  if(!scl)
                     begin
                        link_sda <= 1;
                        sda_buf <= 1;
                        state <= 10;
                     end

                  else
                     state <= 9;
               end
            10: begin//发送启动信号
                  if(scl)
                     begin
                        sda_buf <= 0;
                        state <= 11;
                        eeprom_data <= 8'b1010_0001;
                     end
                  else
                     state <= 10;
               end
            11:begin//发送器件地址
                  if(!scl && count < 8)
                     begin
                        count <= count + 1;
                        eeprom_data <= {eeprom_data[6:0], eeprom_data[7]};
                        sda_buf <= eeprom_data[7];
                     end
                  else if(count == 8 && !scl)
                     begin
                        count <= 0;
                        link_sda <= 0;
                        state <= 12;
                     end

                  else
                     state <= 11;
               end
            12:begin//检测应答信号
                  if(scl)
                     begin
                        if(!sda)
                           state <= 13;
                     end
               end
            13:begin//接收数据
                  if(scl && count < 8)
                     begin
                        count <= count + 1;
                        data[7 - count] <= sda;
                     end
                  else if(count == 8)
                     begin
                        count <= 0;
                        state <= 14;
                        sda_buf <= 1;
                        link_sda <= 1;
                     end
                  else
                     state <= 13;
               end

            14:begin//发送非应答信号
                  if(scl)
                     begin
                        sda_buf <= 1;
                        state <= 15;
                     end
                  else
                     state <= 14;
               end
            15:begin//发送停止信号
                  if(!scl)
                     sda_buf <= 0;
                  else if(scl)
                     begin
                        sda_buf <= 1;
                        state <= 0;
                        rd <= 0;
                        led <= 4'b0000;
                     end
                  else
                     state <= 15;
               end
            default: state <= 0;
         endcase
      end
   end

endmodule
```

一定要注意在发送数据前，先打开总线开关，即取得总线控制权，数据发送完毕后关闭总线开关，即释放总线控制权，然后再检测应答信号；读数据的时候要先释放总线控制权，读完后取得总线控制权，拉高SDA，即发送非应答信号，最后在SCL高电平期间拉低SDA，即发送停止信号。

* **测试代码如下：**

```
module tb;

   reg sys_clk;
   reg rst_n;
   reg key_wr;
   reg key_rd;

   wire scl;
   wire [7:0] data;
   wire led;

   wire sda;

   always #10 sys_clk = ~sys_clk;

   iic_wr_ctrl uut (
      .sys_clk(sys_clk),
      .rst_n(rst_n),
      .key_wr(key_wr),
      .key_rd(key_rd),
      .sda(sda),
      .scl(scl),
      .data(data),
      .led(led)
   );


   initial begin

      sys_clk = 0;
      rst_n = 0;
      key_wr = 0;
      key_rd = 0;
      #100;
      rst_n = 1;
      #120;
      key_wr = 1;
      #5020;
      key_wr = 0;
      #400020;
      key_rd = 1;
      #5020;
      key_rd = 0;

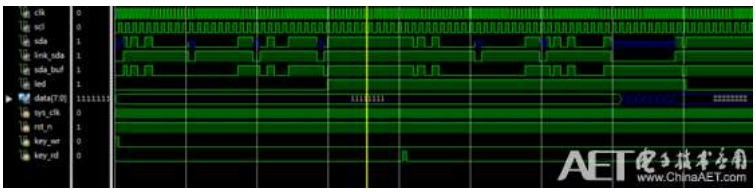
   end

endmodule
```

仿真的时候由于没有EEPROM返回的应答信号，所以可以在源代码上做一点修改，如下所示，略过检测应答信号，直接跳转状态即可

```
7: begin
/*    if(scl)
         begin
            if(!sda)
               state <= 8;
         end*/
         state <= 8;
   end
```

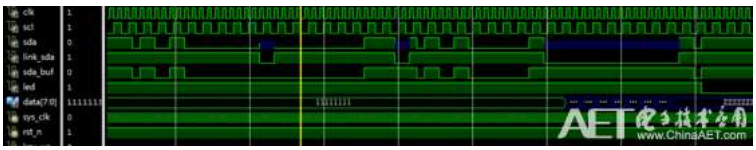
* **仿真图如下：**



写时序：



读时序：



说明：蓝线部分为总线释放了控制权之后，所以处于高阻态；数据的改变、数据总线的放开都是在 SCL 低电平的

中间位置实现的。

* IIC 协议的时序非常严密，所以对我们学习时序的实现技巧和方法有很大的帮助，写这篇博客花费了很长时间，一方面，这是我写的第二篇技术博客，不是特别熟练，所以难免会出现一些错误或者不完善的地方，如有发现，还请劳烦留言指正，必定会及时改正，另一方面，虽然花费了很长时间，但是对于此次设计的理解较为深刻，所以欢迎有共同兴趣的朋友留言探讨，一起学习，共同进步！

* 技术只有多交流才能有更大的进步~

* 欢迎留言探讨~

« 上一篇：基于FPGA的多功能数字时钟设计
» 下一篇：32位超前进位加法器的设计

代码语言

字数统计

提交



Bryan
2017-11-20 13:27:25

 不错，很详细

回复

米客-显示驱动专家
2016-11-26 21:01:18

可以写个N字节的，标准IIC协议包，方便大伙使用哈

回复

y18810459972
2016-11-17 19:15:31

IIC 在很多地方都有用到，不错！

回复

mclaren1985
2016-11-04 16:21:47

不错，很详细

回复



T-Tang
2016-10-19 13:30:10

@swenwong



回复



swenwong

2016-10-18 16:17:13

不错不错，整理的很详细么

回复

« 1 »

网站相关

关于我们

联系我们

投稿须知

广告及服务

内容许可

广告服务

杂志订阅

会员与积分

积分商城

会员等级

会员积分

VIP会员

关注我们

Copyright © 2019 电子技术应用 版权所有 京ICP备10017138号